

Lab: Validating the ShopSphere Microservices Platform through Comprehensive Automated Testing

Business Scenario

The ShopSphere platform has successfully evolved into a production-ready cloud-native microservices ecosystem consisting of independently deployable Spring Boot services. The platform includes centralized configuration using **Spring Cloud Config Server**, dynamic service registration through **Eureka Service Discovery**, centralized request routing using an **API Gateway**, reusable JWT-based authentication and authorization through a **Common Security Module**, synchronous and asynchronous inter-service communication, containerized deployment using Docker, application logging, and runtime monitoring.

Before the platform can be released for production use, the Quality Assurance (QA) and Engineering teams must establish a comprehensive automated testing strategy to ensure the reliability, maintainability, and security of every business service.

The architecture team mandates that every layer of the application—including the Service layer, Controller layer, Security layer, and REST APIs—must be validated using automated tests. Individual business components should be tested in isolation using unit testing, while complete application workflows should be verified through integration and API testing.

To ensure proper isolation between development, testing, and production environments, the engineering team must maintain a dedicated testing configuration for the ShopSphere platform. All automated tests must execute using a separate **Spring Boot test profile** that loads its configuration from an **application-test.properties** file. This configuration should contain environment-specific settings required exclusively for testing, including datasource configuration, application properties, and other test-specific settings. Under no circumstances should automated tests rely on or modify the default application configuration used for development or production environments.

Business services should be tested independently without relying on actual repository implementations by using mocked dependencies. Repository interactions should be simulated using Mockito to validate business logic without requiring direct database access during unit testing.

The engineering team will also use Spring Boot's testing framework to load the application context and validate the integration between application components. REST APIs should be tested using **TestRestTemplate** to verify request processing, response generation, and CRUD functionality within the Spring Boot environment.

In addition, the QA team requires comprehensive API validation using **Rest Assured** and the **Java HTTP Client**. These tools will be used to verify request handling, response structures, HTTP status codes, request headers, JSON payloads, and various positive and negative business scenarios.

Since the ShopSphere platform exposes secured REST APIs protected using **JWT-based authentication, Role-Based Access Control (RBAC), Authority-Based Access Control, and Method-Level Security**, the testing strategy must also validate the complete security implementation. Authentication, authorization, protected resources, and standardized security error responses must all be verified through automated test cases.

The enhanced testing strategy should ensure that every business capability implemented throughout the ShopSphere platform is validated through repeatable automated tests while maintaining complete isolation from development and production environments.

TODOs

JUnit 5

Introduction to JUnit 5

- Understand the purpose of automated unit testing.
- Configure JUnit 5 within the Maven project.
- Organize application test classes following standard project structure.

Writing Test Cases

Create unit test cases for application components.

Validate:

- Business logic
- CRUD operations
- Business rules
- Expected outputs
- Boundary conditions

Test Lifecycle

Understand and implement JUnit 5 lifecycle annotations.

Configure appropriate methods for:

- Test initialization
- Test execution

- Resource cleanup
-

Assertions

Use JUnit assertions to validate:

- Expected values
 - Object equality
 - Boolean conditions
 - Null and non-null values
 - Collection contents
-

Testing Exception Scenarios

Write test cases for:

- Business exceptions
 - Validation failures
 - Resource not found exceptions
 - Invalid business requests
-

Parameterized Tests

Create parameterized test cases for multiple input values.

Validate:

- Valid input
 - Invalid input
 - Boundary conditions
-

Positive and Negative Test Cases

Design test cases covering:

Positive Scenarios

- Successful business execution

- Valid requests
- Correct business results

Negative Scenarios

- Invalid input
 - Validation failures
 - Exception scenarios
 - Business rule violations
-

Mockito

Introduction to Mockito

- Understand the purpose of mocking.
 - Compare mocked testing with real object testing.
 - Configure Mockito within the project.
-

Mocking Dependencies

Implement mocked unit tests using:

- `@Mock`
- `@InjectMocks`

Mock dependencies including:

- Repository layer
 - External service dependencies
-

Stubbing Method Responses

Configure mocked responses for:

- Successful execution
 - Empty responses
 - Exception scenarios
-

Verifying Method Calls

Verify:

- Repository method invocation
 - Number of method executions
 - Expected interactions
 - Unnecessary interactions
-

Testing Service Layer

Test business services using mocked repositories.

Validate:

- Business rules
 - CRUD operations
 - Repository interaction
 - Exception handling
-

Spring Boot Testing

Spring Boot Testing Support

- Configure Spring Boot Test.
 - Load the Spring Application Context.
 - Validate bean creation.
 - Verify dependency injection.
-

Service Layer Testing

Test service-layer components using the Spring context.

Validate:

- Spring-managed beans
- Business functionality
- Dependency injection

Controller Layer Testing

Test REST controllers.

Validate:

- Request processing
- ResponseEntity generation
- HTTP status codes
- Standard API response structure

Unit Testing vs Integration Testing

Understand the differences between:

- Unit Testing
- Integration Testing

Identify appropriate use cases for each testing strategy.

TestRestTemplate

Configure TestRestTemplate.

Test REST APIs including:

- GET
- POST
- PUT
- DELETE

Validate:

- HTTP status codes
 - ResponseEntity
 - Response payload
 - CRUD functionality
-

Test Configuration

Test Environment

Create a dedicated testing environment for the ShopSphere platform.

Configure a separate Spring Boot test profile using an **application-test.properties** file.

The test configuration should:

- Be isolated from the default application configuration.
 - Contain test-specific application properties.
 - Support automated unit and integration testing.
 - Prevent tests from affecting development or production environments.
-

Test Profile

Configure automated tests to execute using the **test profile**.

Ensure that:

- The **application-test.properties** file is loaded during test execution.
 - Spring Boot uses the test-specific configuration.
 - Environment-specific properties are isolated from the default application.
-

Test Database

Configure the testing environment to use a dedicated datasource.

Ensure that:

- Test data is isolated from development and production data.
 - Database configuration is managed through **application-test.properties**.
 - Integration tests execute using the configured test datasource.
-

Spring Boot Test Configuration

Configure Spring Boot test classes to use the testing profile.

Verify that:

- Spring Boot loads the test configuration successfully.

- Test classes execute using the **application-test.properties** file.
 - Integration tests run independently of the default application configuration.
-

API Testing

API Testing Overview

Understand automated REST API testing.

Review:

- Request validation
 - Response validation
 - Error validation
-

Rest Assured

Configure Rest Assured using Maven.

Implement API tests using:

- Given
 - When
 - Then
-

HTTP Request Testing

Test REST APIs using:

- GET
- POST
- PUT
- DELETE

Validate:

- Status codes
- Headers
- JSON responses

- Response body
 - Request body
 - Path parameters
 - Query parameters
-

Positive and Negative API Testing

Validate:

Positive Scenarios

- Successful CRUD operations
- Valid requests
- Expected responses

Negative Scenarios

- Invalid requests
 - Validation failures
 - Business exceptions
 - Resource not found
-

Java HTTP Client

Understand Java HTTP Client.

Implement:

- Synchronous requests
- Asynchronous requests

Compare Java HTTP Client with:

- TestRestTemplate
- Rest Assured

Identify suitable use cases for each.

Testing Secured APIs

Authentication Testing

Validate:

- Successful login
 - Invalid credentials
 - Invalid JWT
 - Expired JWT
 - Missing JWT
-

Authorization Testing

Verify:

- Role-Based Access Control
 - Authority-Based Access Control
 - Method-Level Security
-

Public and Protected APIs

Test:

- Public APIs
- Protected APIs

Verify that only authorized users can access secured resources.

Security Validation

Validate:

- **401 Unauthorized**
 - **403 Forbidden**
 - Standardized security error responses
-

API Security Testing

Test secured APIs using:

- Postman
- TestRestTemplate
- Rest Assured

Validate JWT authentication and authorization behavior.

Security Test Checklist

Verify:

- JWT generation
 - JWT validation
 - Authentication
 - Authorization
 - Public endpoint accessibility
 - Protected endpoint security
 - Role validation
 - Authority validation
 - Invalid token handling
 - Expired token handling
-

Testing Strategy

Design a comprehensive automated testing strategy covering:

- Unit Testing
- Service Layer Testing
- Repository Mocking
- Spring Boot Integration Testing
- Controller Testing
- REST API Testing
- Security Testing
- Positive Test Scenarios

- Negative Test Scenarios
- Test Environment Configuration

Ensure that every feature implemented throughout the ShopSphere platform is validated using automated tests executing within the dedicated Spring Boot test profile.